

HydroBytes

Water Monitoring System

Technical Report

A Smart Water Distribution and Leak Detection Platform for
Real-Time Telemetry, Area-Wise Analytics, Alerting,
and Admin-Based Infrastructure Management

Project Repository: [HydroBytes](#)

Contents

1	Executive Summary	2
2	System Architecture	2
2.1	High-Level Architecture	2
2.2	Directory Structure	2
3	Technology Stack	4
4	Database Design	4
4.1	Main Water Database	4
4.2	Admin Credential Database	5
4.3	Entity Relationship Diagram	5
5	Backend API Design	5
5.1	Endpoint Summary	6
5.2	Authentication Flow	6
6	Telemetry Generation and Leak Detection	7
6.1	Sensor Simulation	7
6.2	Leak Detection Rules	7
7	Analytics Module	8
8	Frontend Architecture	8
8.1	Admin Dashboard	8
8.2	Management Page	8
8.3	User Flow Diagram	9
9	Testing Strategy	9
10	Deployment and Configuration	9
10.1	Development Startup	9
10.2	Environment Variables	10
11	Future Enhancements	10
12	Conclusion	11

1 Executive Summary

HydroBytes is a FastAPI-based smart water distribution and leak detection system. It simulates water sensor telemetry for registered areas and pipelines, stores the readings in SQLite, evaluates leak conditions using deterministic business rules, and exposes REST endpoints for telemetry, analytics, alert handling, and infrastructure management.

The project also includes a browser-based admin interface. The admin can log in, view a water monitoring dashboard, inspect open alerts, resolve alerts, and manage areas and pipelines through a separate management page.

Key Metrics at a Glance

Backend Framework	FastAPI with Uvicorn ASGI server
Database	SQLite file database for water data and separate admin credentials
API Routers	Authentication, telemetry, alerts, analytics, infrastructure
Core Entities	Areas, pipelines, telemetry readings, alerts, admin users
Leak Rules	Low pressure, flow-consumption mismatch, sudden pressure drop
Frontend Pages	Login/dashboard page and area/pipeline management page
Security Feature	PBKDF2-SHA256 password hashing through Passlib

2 System Architecture

2.1 High-Level Architecture

HydroBytes follows a layered architecture. The frontend is a static HTML/CSS/JavaScript interface served by FastAPI. The API layer exposes REST endpoints. The service layer generates telemetry, detects leaks, and computes analytics. The persistence layer stores water operations data in SQLite and admin login data in a separate database.

2.2 Directory Structure

```
Water-master/  
|-- main.py           # FastAPI app, static frontend,  
    router registration
```

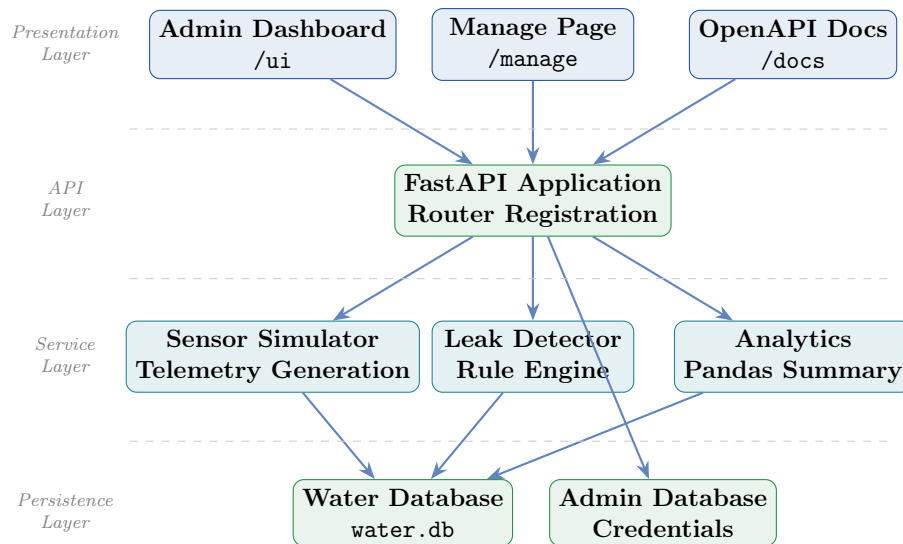


Figure 1: HydroBytes High-Level Architecture

```

|-- database.py           # SQLite schema, CRUD helpers, seed
                           data
|-- admin_database.py     # Admin credential DB and password
                           hashing
|-- sensor_simulator.py  # Synthetic water telemetry
                           generator
|-- leak_detector.py      # Leak detection business rules
|-- analytics.py         # Area and pipeline analytics
|-- schemas.py           # Pydantic request/response models
|-- logger_config.py     # Console and file logging
                           configuration
|-- requirements.txt      # Python dependencies
|-- SECURITY_SOLUTION.md  # Explanation of secure admin
                           authentication
|-- frontend/
|   |-- index.html       # Login and admin dashboard
|   |-- manage.html      # Area and pipeline management page
|   |-- script.js        # Dashboard login, API calls,
                           rendering
|   |-- manage.js        # Management page API calls and
                           charts
|   |-- style.css        # Shared visual styling
|-- routes/
|   |-- auth.py          # POST /login
|   |-- telemetry.py     # /generate-data, /water-data, /
                           logs
|   |-- alerts.py        # /alerts

```

```

| |-- analytics_routes.py # /analytics
| '-- infrastructure.py # /areas and /pipelines
'-- tests/
    |-- conftest.py
    |-- test_generation.py
    |-- test_analytics.py
    '-- test_leaks.py

```

Listing 1: Project Directory Layout

3 Technology Stack

Table 1: HydroBytes Technology Stack

Layer	Technology	Role / Notes
Backend Framework	FastAPI	REST API, request validation, OpenAPI docs
ASGI Server	Uvicorn	Local development and deployment server
Scheduling	APScheduler	Periodic background telemetry generation
Database	SQLite	File-based local persistence
Validation	Pydantic	Request and response schemas
Analytics	Pandas	Aggregation of flow, pressure, consumption, and leaks
Security	Passlib PBKDF2-SHA256	Secure admin password hashing and verification
Frontend	HTML, CSS, Vanilla JavaScript	No build step, served as static files
Testing	Pytest	Unit and integration-style tests with isolated test database
Logging	Python logging	Console output and logs/system_logs.log

4 Database Design

4.1 Main Water Database

The water database stores the operational entities of the system: registered areas, pipelines, generated telemetry, and leak alerts. The schema is created automatically when the application starts.

Table 2: Main Database Tables

Table	Purpose
areas	Stores water distribution areas such as Kothrud or Hinjewadi
pipelines	Stores pipelines belonging to areas
telemetry	Stores generated sensor readings: flow, pressure, consumption, timestamp
alerts	Stores leak alerts generated from telemetry readings

4.2 Admin Credential Database

Admin credentials are stored separately from the water telemetry database. The default admin account is created only when no admin record exists. Passwords are never stored as plain text; they are hashed using PBKDF2-SHA256.

Security Design

- Admin login is validated through the backend endpoint `POST /login`.
- Password hashing is handled in `admin_database.py`.
- The browser stores only a local authentication flag, not the real password.
- The frontend no longer contains hardcoded admin credentials.

4.3 Entity Relationship Diagram

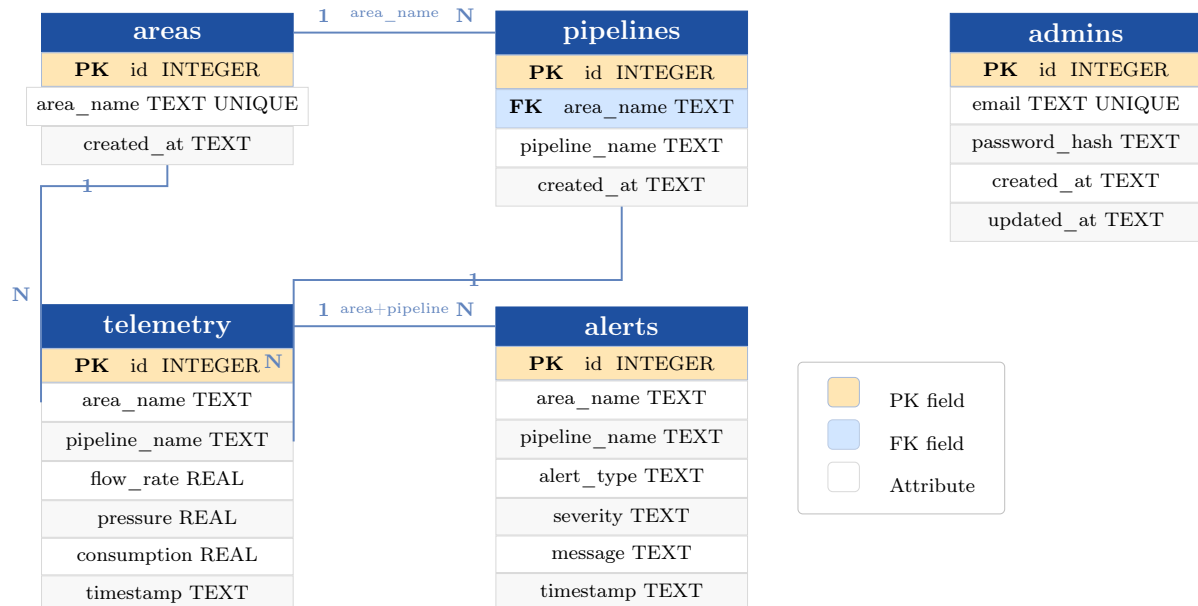


Figure 2: Entity-Relationship Diagram HydroBytes Database Schema

5 Backend API Design

5.1 Endpoint Summary

Table 3: REST API Endpoints

Method	Endpoint	Description
GET	/	Basic API health response
GET	/ui	Serves the admin login and dashboard page
GET	/manage	Serves the area and pipeline management page
POST	/login	Validates admin email and password
POST	/generate-data	Generates one simulated water telemetry reading
GET	/water-data	Lists telemetry readings, optionally filtered by area and pipeline
GET	/logs	Returns recent application log lines
GET	/alerts	Lists stored leak alerts
DELETE	/alerts/{alert_id}	Resolves an alert by deleting it
GET	/analytics	Returns computed analytics summary
GET	/areas	Lists registered water distribution areas
POST	/areas	Creates a new area
DELETE	/areas	Deletes an area and related records
GET	/pipelines	Lists pipelines, optionally filtered by area
POST	/pipelines	Creates a pipeline under an area
DELETE	/pipelines	Deletes a pipeline and related records

5.2 Authentication Flow

1. Admin opens /ui.
2. The login form submits email and password to `POST /login`.
3. FastAPI calls `validate_admin_credentials()` from `admin_database.py`.
4. Passlib verifies the plain password against the stored hash.
5. If valid, the frontend sets a local auth flag and shows the dashboard.
6. Logout clears the auth flag, resets the login form, and returns to the login page.

```
POST /login
Content-Type: application/json
```

```
{  
  "email": "admin@gmail.com",  
  "password": "pmc"  
}
```

Listing 2: Login Request Example

6 Telemetry Generation and Leak Detection

6.1 Sensor Simulation

The simulator generates readings for a selected area and pipeline, or picks a registered pipeline automatically. Each reading includes:

- flow_rate
- pressure
- consumption
- timestamp
- area_name and pipeline_name

The generated data is inserted into the telemetry table. After insertion, the system passes the reading to the leak detector and creates an alert when a rule is triggered.

6.2 Leak Detection Rules

Table 4: Leak Detection Rule Set

Rule	Severity	Condition
Low Pressure	Critical	Pressure below 40 PSI
Flow Mismatch	Warning	Flow rate exceeds consumption by more than 30 units
Pressure Drop	Critical	Pressure drops by more than 20 percent compared to previous reading

Alert Creation Logic

If multiple rules are triggered by the same telemetry reading, HydroBytes combines the messages and keeps the highest severity. The alert is linked to the telemetry reading so the dashboard can show whether a reading created an alert.

7 Analytics Module

The analytics module uses Pandas to compute operational summaries from telemetry and alert data. It returns both global statistics and area-wise breakdowns.

Table 5: Analytics Output Fields

Metric	Description
<code>total_water_flow</code>	Total generated water flow across all telemetry rows
<code>total_consumption</code>	Total consumption across all telemetry rows
<code>water_loss</code>	Difference between water flow and consumption
<code>average_pressure</code>	Average pressure from all readings
<code>leak_count</code>	Number of generated alerts
<code>area_wise_statistics</code>	Per-area averages, leak counts, and pipeline-level stats

8 Frontend Architecture

8.1 Admin Dashboard

The dashboard is served at `/ui`. It contains the login form and the post-login dashboard in a single HTML file. JavaScript controls which section is visible based on the local authentication flag.

Dashboard features:

- Login form for admin authentication.
- Metric cards for total flow, consumption, average pressure, alerts, areas, and pipelines.
- Open alert list with a resolve button.
- Area summary list based on analytics output.
- Navigation button to the management page.
- Logout button that clears the local auth flag and resets the form.

8.2 Management Page

The management page is served at `/manage`. It allows the admin to create and remove areas and pipelines, view data by selected area, and inspect area-level charts.

Management features:

- Add area.
- Remove area.
- Add pipeline under an area.
- Remove pipeline.
- Select an area and view its pipelines and latest water readings.

- View area water balance and leak priority visualizations.

8.3 User Flow Diagram

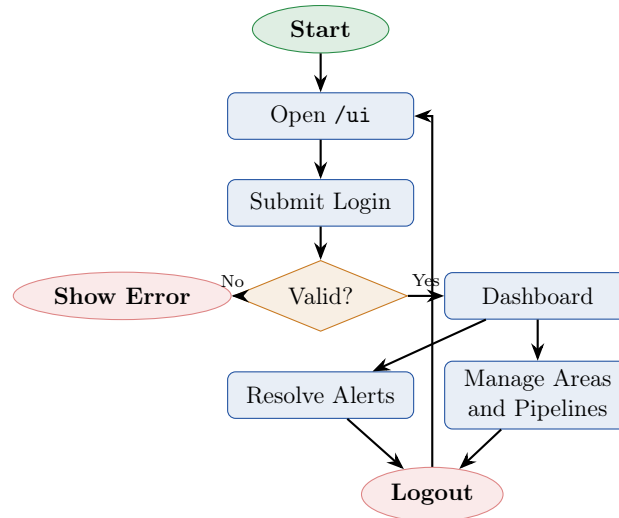


Figure 3: Admin User Flow

9 Testing Strategy

The project includes a Pytest suite focused on core logic. The tests use an isolated database path so local development data is not damaged during test execution.

Table 6: Test Suite Summary

Test Module	Coverage Area
<code>test_generation.py</code>	Verifies telemetry generation and database insertion
<code>test_analytics.py</code>	Verifies analytics totals, averages, and breakdowns
<code>test_leaks.py</code>	Verifies leak rules for low pressure, high flow mismatch, and pressure drop
<code>conftest.py</code>	Provides test database setup and cleanup fixtures

10 Deployment and Configuration

10.1 Development Startup

```

# 1. Install dependencies
pip install -r requirements.txt

# 2. Start the FastAPI application
uvicorn main:app --reload

```

```
# 3. Open the admin UI
http://127.0.0.1:8000/ui

# 4. Open the API documentation
http://127.0.0.1:8000/docs
```

Listing 3: Development Startup

10.2 Environment Variables

Table 7: Environment Configuration

Variable	Default	Purpose
WATER_DB_PATH	database/water.db	Path for operational water database
WATER_LOG_PATH	logs/system_logs.log	Path for application log file

Production Checklist

- Use HTTPS behind a reverse proxy such as Nginx or Caddy.
- Move from local SQLite to a managed database if concurrent usage increases.
- Add JWT or signed server-side sessions instead of a local browser auth flag.
- Rate-limit POST /login to reduce brute-force attempts.
- Keep generated files such as databases, logs, caches, and virtual environments out of Git.
- Add backup automation for telemetry and admin credential databases.

11 Future Enhancements

- Add role-based accounts for multiple administrators.
- Add a change-password endpoint and frontend form.
- Replace local authentication flag with JWT access tokens or server-side sessions.
- Add real-time updates through WebSockets or Server-Sent Events.
- Add map-based visualization for areas and pipeline locations.
- Add alert acknowledgement history instead of deleting resolved alerts.
- Add deployment configuration with Docker and environment-based settings.

12 Conclusion

HydroBytes demonstrates a complete smart water monitoring workflow: infrastructure registration, synthetic telemetry generation, leak detection, analytics, alert resolution, and secure admin login. Its modular design separates API routes, database helpers, simulation logic, leak detection, analytics, logging, and frontend rendering. This makes the project understandable, testable, and suitable for future extension into a production-ready municipal water monitoring platform.